

Dominando CSS Grid

Guia Prático e Objetivo Baseado na Aula

1. O Básico do CSS Grid

- **Inicialização:** Ative o sistema no elemento pai utilizando `display: grid;`.
- **Estruturação Inicial:** Defina a quantidade e a dimensão de linhas e colunas pelas propriedades `grid-template-rows` e `grid-template-columns`.
- **Unidade Fracional (fr):** Utilize fragmentos (`fr`) para distribuir proporcionalmente o espaço livre (exemplo: `1fr 2fr 1fr`).
- **Função de Repetição:** Reduza o código repetitivo usando a função `repeat()` (exemplo: `repeat(3, 1fr)` gera três colunas iguais).

2. Responsividade Automática

- **Preenchimento Dinâmico:** A palavra-chave `auto-fill` permite enfileirar o máximo de colunas que o espaço do navegador permitir, quebrando as linhas automaticamente conforme o redimensionamento.
- **Tamanhos Flexíveis:** A função `minmax()` força um tamanho mínimo e permite um tamanho máximo (exemplo: `minmax(320px, 1fr)` impede que a coluna seja menor que 320px, mas deixa crescer até preencher todo o espaço restante).

3. Espaçamentos (Gaps)

- **Distanciamento Uniforme:** Aplique a propriedade `gap` no elemento pai para criar uma distância (calha) constante entre colunas e linhas.
- **Controle Direcionado:** É possível espaçar os eixos individualmente usando `row-gap` (para linhas) e `column-gap` (para colunas).
- **Compatibilidade com Navegadores:** O suporte para `gap` é defasado em versões de navegadores muito antigos (como o Internet Explorer); as alternativas comuns giram em torno de gambiarras com margens (`margin`), ocultamento de extravasamentos (`overflow: hidden`) ou cálculos no CSS.

4. Posicionamento e Expansão de Itens

- **Mapeamento via Linhas:** Movimente filhos do Grid marcando onde eles devem começar e terminar, usando números de linha positivos (ou negativos, de trás para frente).

- **Eixos Específicos:** Altere posições na horizontal com `grid-column-start` e `grid-column-end`, e na vertical com `grid-row-start` e `grid-row-end`.
- **Expansão de Espaço (Span):** Faça elementos ocuparem múltiplas seções utilizando o termo `span` (exemplo: `grid-column: span 2;` faz a célula esticar o equivalente a duas colunas).
- **Sobreposição e Camadas:** Elementos do grid podem ocupar o mesmo espaço intencionalmente. Nestes casos, o `z-index` dita qual fica visualmente sobreposto ao outro.

5. Grid Template Areas

- **Prototipagem Semântica:** Construa a estrutura criando "mapas" no CSS através da propriedade `grid-template-areas`, escrevendo literalmente o formato de tela almejado.
- **Nomeando os Componentes:** Batize cada área que deseja manipular designando uma `grid-area` específica (exemplo: dar o valor `header` ao cabeçalho e `footer` ao rodapé).
- **Declaração de Organização:** Distribua as áreas na regra pai simulando o formato final (exemplo: `"header header" "nav main" "footer footer"`).
- **Aninhamento:** Em um container mapeado com Grid, você pode ter elementos internos contendo outro Flexbox ou Grid, como barras de navegação (`display: flex`) perfeitamente alinhadas.

6. Media Queries (Responsividade Direcionada)

- **Sobrescrita Direta:** Utilize `@media` para interceptar quebras de tela (breakpoints, como `max-width: 768px`) e mudar completamente a grade.
- **Reestruturação Móvel:** Substitua os mapeamentos criados em `grid-template-areas` ou redefina o número de colunas criadas, simplificando os conteúdos para que fiquem enfileirados de cima para baixo em uma só coluna em telas de smartphones.

PRÁTICA

- **Missão do Layout:** Projetar e desenvolver uma interface de gerenciamento analítico (Dashboard) perfeitamente adaptável.
- **Resultado Esperado para Monitores Grandes:** A tela deve se ramificar de forma bem definida. O Cabeçalho (topo) e o Rodapé (fundo) abrangerão a página de ponta a ponta. O meio da tela será bifurcado em duas grandes áreas horizontais contínuas: à esquerda, o Menu Lateral de opções; e à direita (preenchendo a maior fatia da tela), a Superfície Central de Dados.
- **Resultado Esperado na Superfície Central:** Esse bloco de conteúdo exibirá múltiplos painéis isolados (cartões de informações). Estes cartões devem fluir ocupando todo o miolo livre, evitando tamanhos comprimidos. Como cereja do bolo, um cartão principal (gráfico ou aviso crítico) precisa ignorar a sequência convencional das células, exigindo mais fatias na largura e na altura dentro desse mesmo contêiner.
- **Resultado Esperado para Telas Menores (Smartphones):** O fluxo deve se tornar unilinear e vertical. Cabeçalho, Menu de Opções, Bloco de Cartões e Rodapé farão uma sequência única, como um pergaminho contínuo, impedindo que o layout "esprema" informações e quebre a rolagem horizontal do site.
- **Produto a ser alcançado:** Dois arquivos funcionais e puros, um HTML e um CSS. Os scripts devem expor o uso direto de `display: grid`, gerenciamento de fragmentos flexíveis (`fr`), repetições otimizadas (`repeat`), inteligência autoajustável com `minmax()` e `auto-fill`, quebras de seções via `span`, o mapeamento espacial (`grid-template-areas` e `grid-area`) somados a um sistema de `@media query` bem formatado para reordenar tudo no modo smartphone. Tudo coeso através de espaçamentos uniformes (`gap`).